EsaBend IoT ModBusTCP P2

NOTE 1: Some of the features described may not be present on older versions of EsaBend.
NOTE 2: ver. only 58.1 of EsaBend contains all the features described.

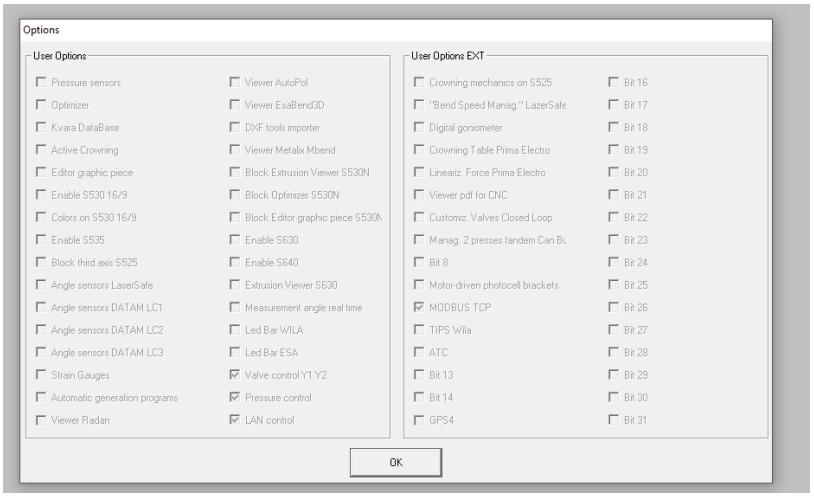
1. Description

EsaBend is enabled to make a series of data visible and receive a series of data and commands from an external supervisor which can reside on the same PC as EsaBend or on a different PC.

To make a series of data visible and receive data and execute commands EsaBend uses a ModBusTCP server; the supervisor must therefore be a ModBusTCP client that reads data and transmits data and commands via the appropriate ModBus registers.

2. Enable feature

The EsaBend ModBusTCP server service is paid feature and must be enabled via KvOption: if the checkbox in "MODBUS TCP" is enabled, the service has been enabled.



Copy file from:

C:\Esabend\Kvara\Exe\NewXMLModBus\ServerModbus.xml

To

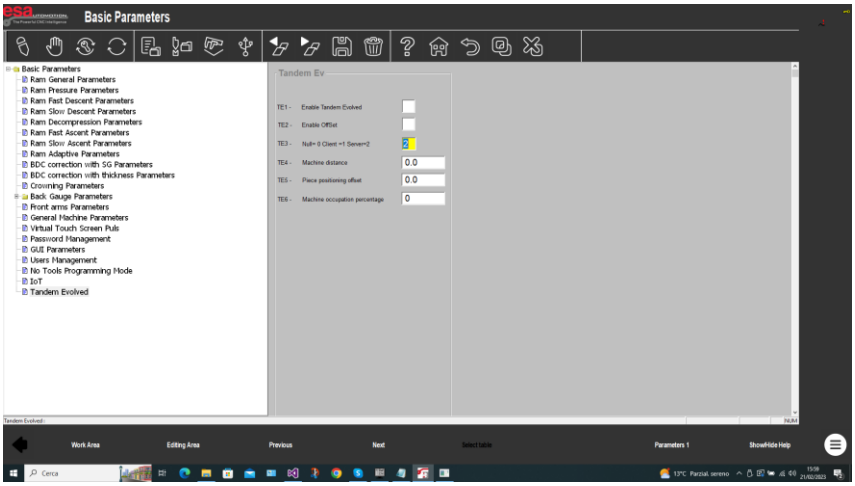
C:\Esabend\Kvara\Exe\ServerModbus.xml

Open ServerModbus.xml and configure

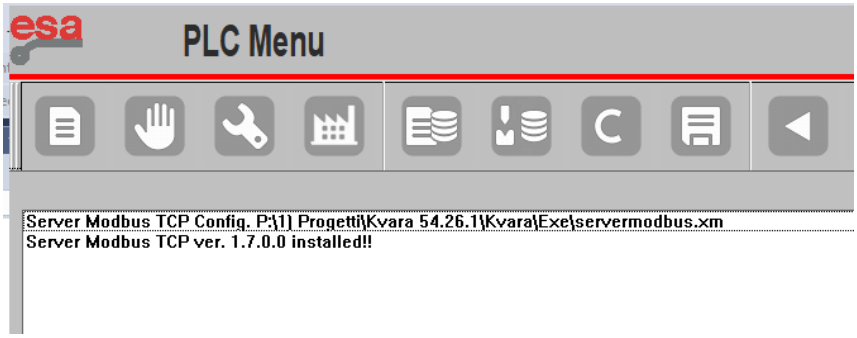
correct IP in:

<IPAddr>127.0.0.1</IPAddr>

In Basic Parameter\Tandem Evolved set the parameter to 2 TE3



To check if the ServerModBusTCP is active, check that the 2 messages have been printed in PLC Menú:



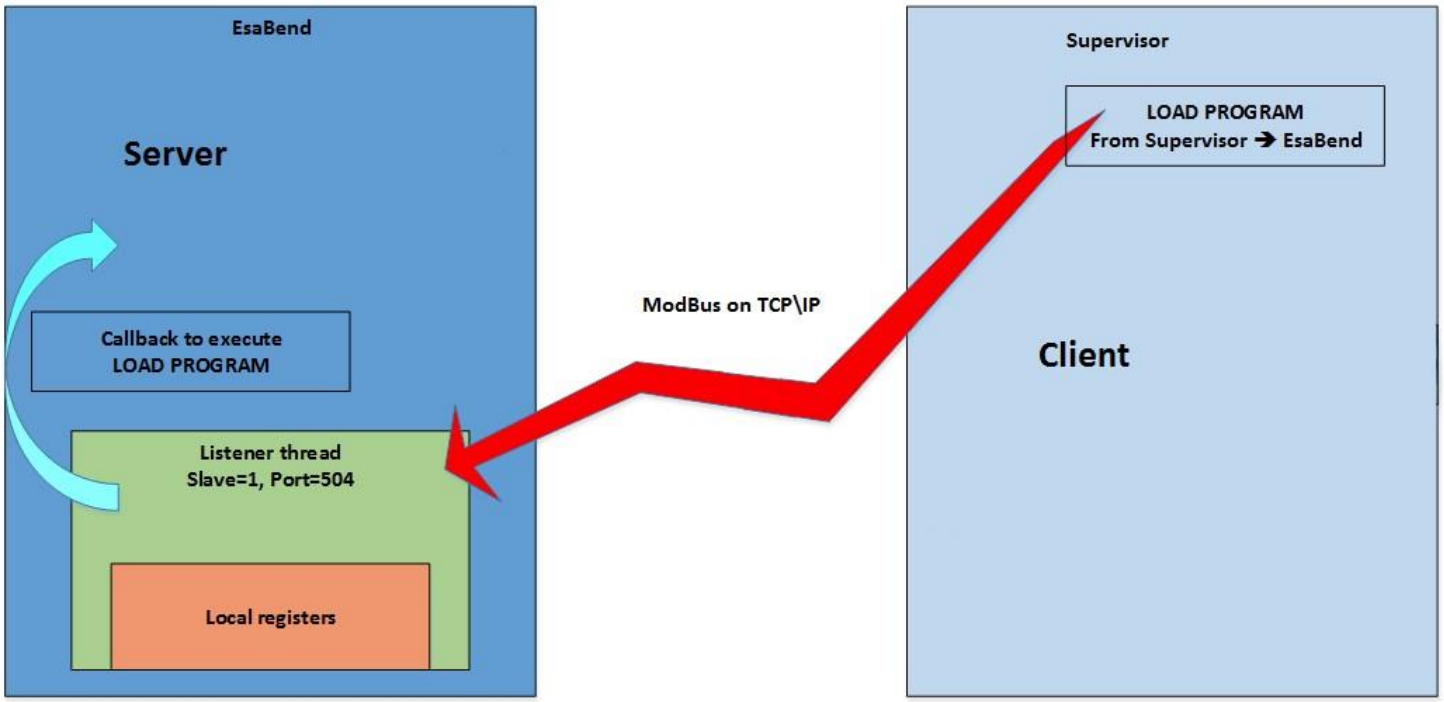
The commands arriving from the supervisor are "heard" by EsaBend from the "Classic" numerical setting pages of the work program.

3. SupervisorClientModBusTCP<->EsaBend architecture

The components that make up the system are:

- EsaBend + ServerModbus.dll
- Supervisor + ClientModBusTCP made by third parties

The data and commands written\read by the ModBusTCP client supervisor are read\written by the ModBusTCP server ServerModbus.dll which then writes\reads in shared memory; the maximum limit of registers read in block is 128; EsaBend reads\writes in shared memory and updates the data and executes commands received from the supervisor; the figure shows the “Load program” request sent by the Supervisor to EsaBend:



4. Binding from\to ModBus register ena EsaBend cnc shared memory register

The binding from\to ModBus register ena EsaBend cnc shared memory register mapped in C:\Esabend\Kvara\Exe\NewXMLModBus\ServerModbus.xml file.

For example:

1) the bending angle of actual step in shared memory is in “progS.angyaS” and in “1253-1256” ModBus register; the binding in ServerModbus.xml file is in input ModBus are:

```
<Field Priority=0 Size=4>
    <MbAddr>1253</MbAddr>
    <ShAddr>progS.angyaS</ShAddr>
</Field>
```

2) the command operating code of EsaBend IoT protocol in shared memory is in “EBCS.wCommandFC” and in “1” ModBus register; the binding in ServerModbus.xml file is in output ModBus are

```
<Field Priority=0 Size=1>
    <MbAddr>1</MbAddr>
    <ShAddr>EBCS.wCommandFC</ShAddr>
</Field>
```

5. Mapping

Address Modbus = number of ModBus register to read EsaBend data, write data to EsaBend and give commands to Esabend

Description = description of the data read or written or of the command executed

N.Register 16 bit = number of ModBus register

Client Output:

Address Modbus	N.Register 16 bit	Description

0	1	**Strobe selection of operating modes,** BIT0 Editor BIT1 Manual BIT2 Semi-automatic BIT3 Automatic
1	1	BIT0 strobe command
2	2	Command opcode
4	1	Given if type byte
5	1	Given if type word
6	1	
7	2	Given if type dword
9	4	Dato se tipo double
53	40	Given if type string

Output EsaBend Server:

Address Modbus	N.Register 16 bit	Description
1024	2	Error code
1026	100	Error description header
1126	100	Error description
1226	1	Acknowledgement operative modes: BIT0 Editor BIT1 Manual BIT2 SemiAutomatic BIT3 Automatic
1227	1	**Active operating mode:** BIT0 Editor mode active BIT1 Manual mode active BIT2 Semiautomatic mode active BIT3 Automatic mode active
1228	1	**Windows popup error code**
1229	2	
1231	1	BIT0 acknowledgement command
1232	1	Type of bend
….	….	….
….	….	…
1965	82	Program name

The data referring to the fields of the current bend are mapped from ModBus register 1232 to register 1965

Address Modbus	N.Register 16 bit	Description
……	……	
1232	1	Type of bend
1233	1	Rotation
1234	1	Number of times the bend must be repeated before passing on to the next one
….	….	…
1253	4	Bending angle required
…….	……	
1965	82	Program name

Look in ServerModbus.xml to all bend data mapped in ModBus register.

6. Command operating codes

```
COMMAND_WORK_DIR                    3001
COMMAND_RESET_WIN_POPUP_ERR         3002
COMMAND_SAVE_PROG                   3003
COMMAND_CONNECT_DISAB_MOUSE_MACHINE2  3004
COMMAND_CONNECT_MACHINE2            3005
COMMAND_DISCONNECT_MACHINE2         3006
COMMAND_STOP                        3007
COMMAND_START                       3008
COMMAND_LOAD_WORK_LIST              3009
COMMAND_DELETE_PROG                 3010
COMMAND_COPY_PROG                   3011
SHUT_DOWN                           3012

COMMAND_NEXT                        3013
COMMAND_PREV                        3014
COMMAND_NEW_PROG_NUM                3015
COMMAND_SUPPORT                     3016
COMMAND_DUTCH                       3017
COMMAND_CHANGE_SECTION              3018
COMMAND_INCREMENTAL                 3019
COMMAND_COINING                     3020
COMMAND_UPDATE_PROG                 3021
COMMAND_DELETE_SECTION              3022
COMMAND_CHANGE_PRESS_MODE           3023
COMMAND_ACTBOMB                     3024
COMMAND_SAVE_AS_PROG                3025
COMMAND_BUMPING                     3026
COMMAND_DEL_BEND                    3027
COMMAND_CONIC                       3028
COMMAND_NUM_BEND_SEC                3029
COMMAND_LOAD_PROG                   3030
COMMAND_LEARN_Y                     3031
COMMAND_LEARN_Z                     3032
COMMAND_CHANGE_PROGRAM              3033
COMMAND_CHANGE_PROGRAM_BG           3034
COMMAND_CORR_BY_SEC                 3035
COMMAND_CORR_BY_PIECE               3036
COMMAND_CHANGE_BEND_METHOD                3037
COMMAND_CHANGE_BEND_METHOD_EQUAL_ANGLES   3038
COMMAND_CHANGE_BEND_METHOD_ALL_PIECE      3039
COMMAND_LEARN_BEND_SENS_ANG               3040
COMMAND_LEARN_BEND_EQUAL_ANGLES_SENS_ANG  3041
COMMAND_CHANGE_OFFSET_POS_PEZZO           3042
COMMAND_CHANGE_OFFSET_POS_PEZZO_BY_SEC    3043
COMMAND_CHANGE_OFFSET_POS_PEZZO_BY_PIECE  3044
COMMAND_RUN_LASERCHECK                    3045
COMMAND_INIT_COMM_LASERCHECK              3046
COMMAND_SEND_BEND_PARAM_LASERCHECK        3047
COMMAND_SEND_SET_TOOL_LASERCHECK          3048
COMMAND_SCAN_ANGLES                       3049
COMMAND_SEND_SET_LASER_ON_OFF_LASERCHECK  3050
COMMAND_SEND_CONT_MEAS_ON_OFF_LASERCHECK  3051
COMMAND_SET_FRAME_RATE_LASERCHECK         3052
COMMAND_ECO_REQUEST_LASERCHECK            3053
COMMAND_SAUTO_AUTO_GRAFICO                3054
COMMAND_NUM_BEND_SEC_GRAFICO              3055
```

7. Command operating codes that change program and bend values

```
HD_PIECEP        0     // programmed pieces
HD_PIECEM        1     // pieces made

HD_MEASURES      11
HD_DEVELOP       12
HD_NAME_PRG      13
HD_ACT_BEND      14
HD_ACT_SECT      15
HD_NOME                          16      // program name
HD_NUM_BEND_SEC                         17// currently selected bend number
HD_NUMDATI                       18          // bend data number in double
```

1000 must be added to the following codes:

```
PG_TYPE                                 1000       // type of step
PG_ROT                                  1001       // direction of fold
PG_RIP                                  1002       // number of caland step repetitions.
PG_FUN                                  1003       // dword 20 machine functions
PG_NUMDATIB                              1004       // Number of bend data in bytes
PG_ACCOMP                               1005       // accompanying speed
PG_THICK                                1006       // sheet thickness
PG_TBEND                                1007       // time spent in the fold
PG_CORPIN                               1008       // correction altitude 180
PG_ANGY1                                1009       // angle y1
PG_ANGY2                                1010      // angle y2
PG_X1                                         1011      // double   axis x1

PG_X2                                         1012      // double   axis x2
PG_X3                                         1013      // double   axis x3
PG_X4                                         1014      // double   axis x4
PG_Y1                                         1015      // double   axis y1
PG_Y2                                         1016      // double   axis y2
PG_R1                                         1017      // double   axis r1
PG_R2                                         1018      // double   axis r2
PG_R3                                         1019      // double   axis r3
PG_R4                                         1020      // double   axis r4
PG_Z1                                         1021      // double   axis z1
PG_Z2                                         1022      // double   axis z2
PG_Z3                                         1023      // double   axis z3
PG_Z4                                         1024      // double   axis z4
PG_A1                                         1025      // double   axis auxiliary 1
PG_A2                                         1026      // double   axis auxiliary 2
PG_A3                                         1027      // double   axis auxiliary 3
PG_A4                                         1028      // double   axis auxiliary 4
PG_A5                                         1029      // double   axis auxiliary 5
PG_A6                                         1030      // double   axis auxiliary 6
PG_A7                                         1031      // double   axis auxiliary 7
PG_A8                                         1032      // double   axis auxiliary 8
PG_CORAY1                        1033      // correction angle y1
PG_CORAY2                        1034      // correction angle y2
PG_CORX1                         1035      // correction x1
PG_CORX2                         1036      // correction x2
PG_CORX3                         1037      // correction x3
PG_CORX4                         1038      // correction x4
PG_CORY1                         1039      // correction y1
PG_CORY2                         1040      // correction y2
PG_RINX1                         1041      // retract x1
```

```
PG_RINX2                    1042    // retract x2
PG_RINX3                    1043    // retract x3
PG_RINX4                    1044    // retract x4
PG_VELW                     1045    // working speed
PG_TCP                      1046    // step change time
PG_PMS                      1047    // top dead center
PG_NSTAZ                    1048    // station number
PG_CSENS                    1049    // flag sensor
PG_SENS1                    1050    //
PG_SENS2                    1051    //
PG_SENS3                    1052    //
PG_SENS4                    1053    //
PG_SENS5                    1054    //
PG_SENS6                    1055    //
PG_METODO                   1056    //
PG_SA                       1057
PG_SB                       1058    //
PG_SC                       1059    //
PG_SD                       1060    //
PG_SE                       1061    //
PG_SF                       1062
PG_ALTALETTA                1063
PG_NEW5                     1064
PG_NEW6                     1065
PG_NEW7                     1066
PG_NEW8                     1067
PG_NEW9                     1068
PG_GEN_B1                   1069
PG_GEN_B2                   1070
PG_GEN_B3                   1071
PG_GEN_B4                   1072
PG_GEN_B5                   1073
PG_GEN_B6                   1074
PG_GEN_B7                   1075
PG_GEN_B8                   1076
PG_GEN_B9                   1077
PG_GEN_D1                   1078
PG_GEN_D2                   1079
PG_GEN_D3                   1080
PG_GEN_D4                   1081
PG_GEN_D5                   1082
PG_GEN_D6                   1083
PG_GEN_D7                   1084
PG_GEN_D8                   1085
PG_GEN_D9                   1086
PG_SPEED_DEC_LC             1087
PG_DEC_DISTANCE_LC          1088
PG_FIN_BEND_STATUS_LC 1089
PG_Y1PERDEG_LC        1090
PG_Y2PERDEG_LC        1091
PG_SB_Y1_LC           1092
PG_SB_Y2_LC           1093
PG_ANGY1_FIN_LC       1094
PG_ANGY2_FIN_LC       1095
PG_FORCE_MIS_LC       1096
PG_POSY1_FIN_LC       1097
PG_POSY2_FIN_LC       1098
PG_DB_NUM_STOP_LC     1099
PG_DB_QTA_PCL_LC      1100
PG_SB_IMG100          1101
```

```
PG_WARP_IMG100          1102
PG_CONFIDENCE_IMG100    1103
PG_NUM_TENTA_IMG100     1104
PG_METODO_IMG100        1105
PG_TOLL_IMG100          1106
PG_PREBEND_IMG100       1107
PG_ANG_FIN_IMG100       1108
PG_ANG_PRE_B_IMG100     1109
PG_VEL_SAL_IMG100          1110
PG_DIE                     1111
PG_PUNCH                   1112
PG_SEL_PUNTI_MISURA_LC 1113
PG_QTA_PNT_MIS_Y1_LC    1114
PG_QTA_PNT_MIS_Y2_LC    1115
PG_QTA_PNT_MIS_YB_LC    1116
PG_QTA_PARCH_LASER_LC   1117
PG_SCAN_ANGLE_Y1_LC     1118
PG_SCAN_ANGLE_Y2_LC     1119
PG_NUM_NO_MATCH_Y1_LC   1120
PG_NUM_NO_MATCH_Y2_LC   1121
PG_ANGLE_Y1_FRONT_LC    1122
PG_ANGLE_Y1_BACK_LC     1123
PG_ANGLE_Y2_FRONT_LC    1124
PG_ANGLE_Y2_BACK_LC     1125
PG_PREBEND_PMI          1126
PG_DIE_HOLDER       1127
PG_INTERMEDIATE     1128
PG_PMI_AFTER_PREBEND    1129
PG_LAB_MIN                                  1130
PG_AREA_SIC_X                               1131
PG_AREA_SIC_R                               1132
PG_SENS_PRES_BOMB_ATT      1133
PG_SENS_PRES_FIND_MAX   1134
PG_SENS_PRES_FIND_SPESS         1135
PG_WIDTH                1136
PG_V_DIE                1137
PG_WIDTH_V_DIE          1138
PG_FORCE                1139
PG_CROWNING             1140
PG_MUTEPOINT            1141
PG_INNER_RADIUS         1142
PG_NAME_MATERIAL        1143
PG_NUM_MATERIAL         1144
PG_RESISTANCE           1145
PG_YB                   1146
PG_CORYB                1147
PG_QTA_PARCH_LASER_DX_LC   1148
PG_SENS_ATTIVI             1149
PG_PMI_AFTER_PREBEND_Y2    1150
PG_ROT1                                 1151
PG_ROT2                                 1152
PG_ROT_VEL                 1153
PG_COR_ROT                 1154
PG_ANGY3                        1155      // angle y3
PG_ANGY4                        1156      // angle y4
PG_Y3                           1157      // double   axis y3
PG_Y4                           1158      // double   axis y4
PG_CORY3                        1159      // correction y3
PG_CORY4                        1160      // correction y4
PG_ROT_PUNCH               1161
```

```
PG_ROT_DIE                  1162
PG_OFFSET_POS_PEZZO     1163
PG_CORR_ANG_Y1_LC            1164
PG_CORR_ANG_Y2_LC            1165
PG_CORR_ANG_YB_LC            1166
PG_CORR_QTA_PNT_MIS_Y1_LC    1167
PG_CORR_QTA_PNT_MIS_Y2_LC    1168
PG_CORR_QTA_PNT_MIS_YB_LC    1169
```

8. Protocol

Client Command protocol:

1. Write **data** to one or more ModBus registers address **4** or **5** or **6** or **7** or **9** or **53** (depending on the command)
2. Write **command opcode** in ModBus register address **2**
3. Write **strobe command** (bit0 to 1) in ModBus register address **1** (set strobe)
4. Wait **acknowledgment command** (bit0 to 1) in ModBus register address **1231** (wait acknowledgment go 1)
5. Write **strobe command** (bit0 to 0) in ModBus register address **1** (Reset strobe)
6. Wait **acknowledgment command** (bit0 to 0) in ModBus register address **1231** (wait acknowledgment go 0)

The ModBus registers address **4** or **5** or **6** or **7** or **9** or **53** to write to depends on the data type

For program and bend data, refer to the "Current bend data" chapter.

Client operating mode selection protocol

The EsaBend operating modes that can be selected from the protocol are: Settings, Manual, Semi-Automatic, Automatic.

1. Write to 1 of the appropriate bit depending on the operating mode to be selected in ModBus register address **0** (set strobe)
2. Wait **acknowledgment operatign mode** for the appropriate bit (bitXX to 1) in ModBus register address **1226** (wait acknowledgment go 1)
3. Write to 1 of the appropriate bit depending on the operating mode to be selected in ModBus register address **1** (Reset strobe)
4. Wait **acknowledgment operating mode** for the appropriate bit (bitXX to 0) in ModBus register address **1226** (wait acknowledgment go 0)

Example of Automatic Operating Mode selection

1. Write bit3 to 1 in ModBus register address **0** (set strobe)
2. Wait for bit3 to 1 in ModBus register address **1226** (wait acknowledgment go 1)
3. Write bit3 to 0 in ModBus register address **0** (Reset strobe)
4. Wait for bit3 to 0 in ModBus register address **1226** (wait acknowledgment go 0)

Example of Load Program command

NOTE: Before sending the load program command, select the **Editor** operating mode.

1. Write **program name** to ModBus registers address **53-63**
2. Write **command opcode 3030** in ModBus register address **2**
3. Write **strobe command** (bit0 to 1) in ModBus register address **1**
4. Wait **acknowledgment command bit0 to 1** in ModBus register address **1231**
5. Write **strobe command bit0 to 0** in ModBus register address **1**
6. Wait **acknowledgment command bit0 to 0** in ModBus register address **1231**

Example changes Pieces to Make

NOTE: Before sending data, select the **Editor** operating mode.

1. Write **pices to make** to ModBus registers address **7**
2. Write **command opcode 0** in ModBus register address **2**
3. Write **strobe command** bit0 to 1 in ModBus register address **1**
4. Wait **acknowledgment command bit0 to 1** in ModBus register address **1231**
5. Write **strobe command bit0 to 0** in ModBus register address **1**
6. Wait **acknowledgment command bit0 to 0** in ModBus register address **1231**

Example change Angle

NOTE: Before sending data, select the **Editor** operating mode.

1. Write **angle** to ModBus registers address **9**
2. Write **command opcode 1009** in ModBus register address **2**

3. Write **strobe command** bit0 to 1 in ModBus register address **1**
4. Wait **acknowledgment command bit0 to 1** in ModBus register address **1231**
5. Write **strobe command bit0 to 0** in ModBus register address **1**
6. Wait **acknowledgment command bit0 to 0** in ModBus register address **1231**

8. Debug

The shared memory registers involved for the protocol is defined in defcn_EBCS and to debug the protocol signals just use TksShare.exe and connect the shared memory directors listed below:

```
//EsaBendClientServer
typedef struct {
  //From Client to Server
  word   wMode;              // Strobe selection of operating modes
  word   wCommandFC;         // strobe command
  dword  dwCodeFC;           // Command opcode
  byte   byDataFC;           // Given if type byte
  word   wDataFC;            // Given if type word
  word   wDataFC_b;
  dword  dwDataFC;   // Given if type dword
  double dDataFC;    // Given if type double
  string strDataFC[40];      // Given if type string
  .
  .
  .
  //From Server to Client
  dword   dwErrorCode;       // Error code
  string  strErUITextA[200]; //Error description header

  string  strErUITextB[200]; // Error description
```

```
  word    wAckMode;            // Acknowledgement operative modes
  .
  word    wAckCommandFS;    acknowledgement command
} S_EBCS;
.
S_EBCS EBCS;
```

the mapping ModBus register and shared memory register is:

Client Output:

Address Modbus	N.Register 16 bit	Registers in shared memory	Description
0	1	EBCS.wMode	**Strobe selection of operating modes,** BIT0 Editor BIT1 Manual BIT2 Semi-automatic BIT3 Automatic
1	1	EBCS.wCommand	BIT0 strobe command
2	2	EBCS.dwCodeFC	Command opcode
4	1	EBCS.byDataFC	Given if type byte
5	1	EBCS.wDataFC	Given if type word
6	1	EBCS.wDataFC_b	
7	2	EBCS.dwDataFC	Given if type dword
9	4	EBCS.dDataFC	Dato se tipo double
53	40	EBCS.strDataFC[40]	Given if type string

Output EsaBend Server:

Address Modbus	N.Register 16 bit	Registers in shared memory	Description
1024	2	EBCS.dwErrorCode	Error code
1026	100	EBCS.strErUITextA[200]	Error description header
1126	100	EBCS.strErUITextB[200]	Error description
1226	1	EBCS.wAckMode	Acknowledgement operative modes: BIT0 Editor BIT1 Manual BIT2 SemiAutomatic BIT3 Automatic
1227	1	EBCS.wActualMode	**Active operating mode:** BIT0 Editor mode active BIT1 Manual mode active BIT2 Semiautomatic mode active BIT3 Automatic mode active
1228	1	EBCS.wErrData	**Windows popup error code**
1229	2	EBCS.dwErrValue	
1231	1	EBCS.wAckCommandFS	BIT0 acknowledgement command
1232	1	progS.typeS	Type of bend
….	….	….	….
….	….	….	…
1965	82	nprS	Program name

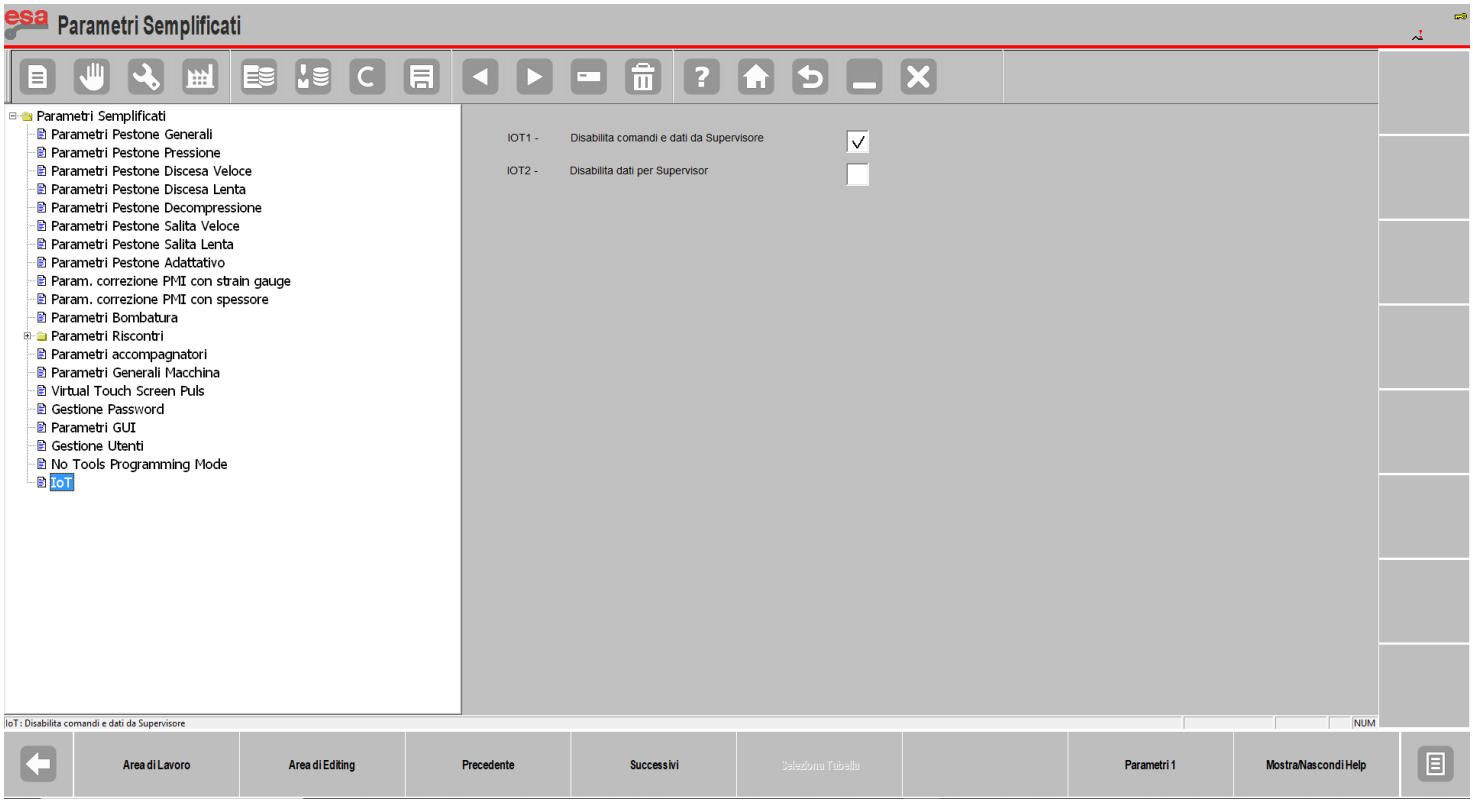
To debug the protocol .

9. Timed reading to avoid disconnection

In the event of a lack of communication, the server disconnects but to avoid this, the client simply needs to read an input register every 10 seconds.

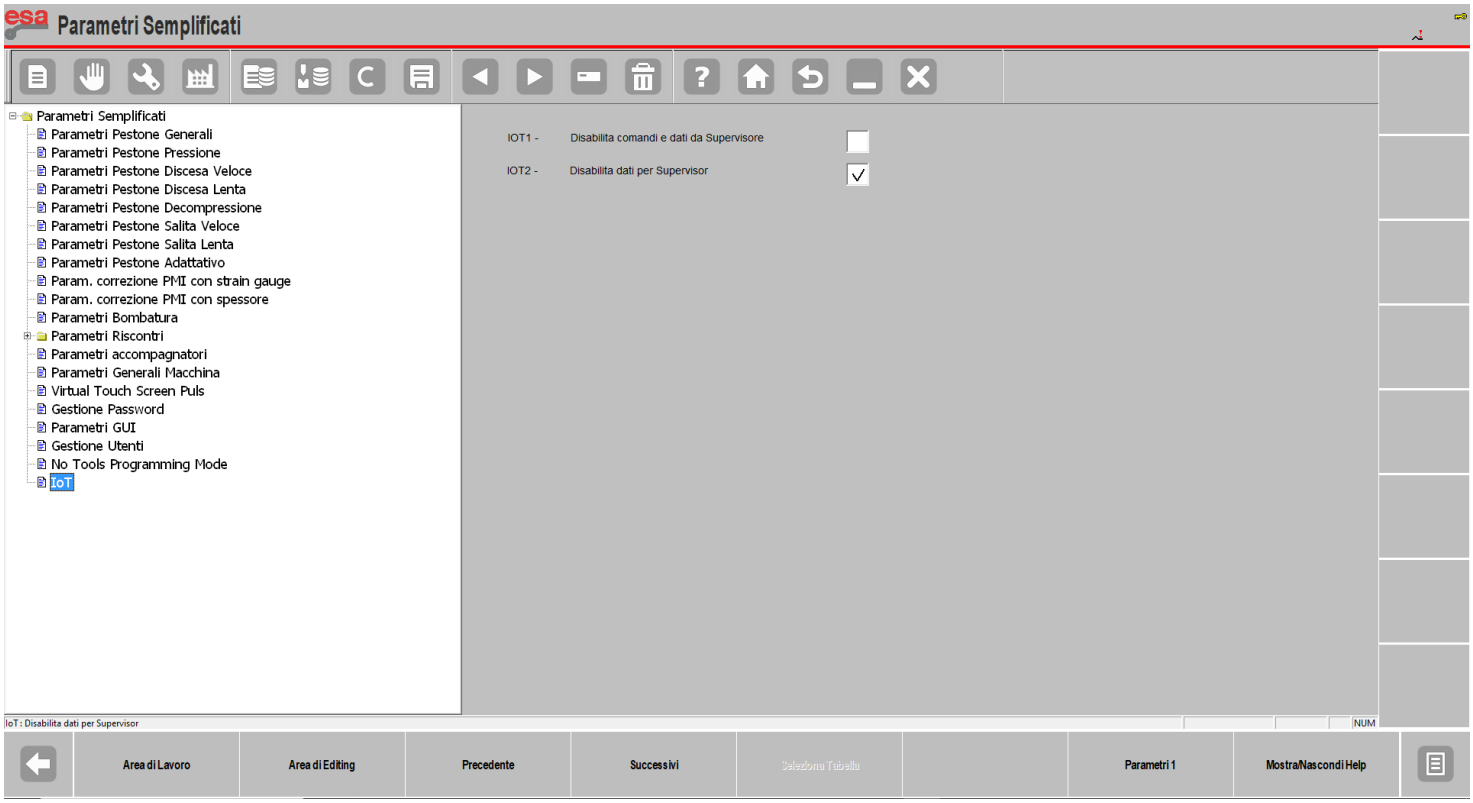
10. Disable commands and data from supervisor

Reception of commands or data from the supervisor is disabled (EsaBend does not execute the command and does not modify the data) if bit C44.15 is set to 1 or the "Disable commands and data from supervisor" parameter is set on the IoT page:



11. Disable data for supervisor

Sending data to the supervisor is disabled (EsaBend does not refresh the data for the supervisor) if bit C44.16 is set to 1 or the "Disable data for supervisor" parameter is set on the IoT page:



12. Data format used in ModBus protocol

Formato WORD 16bit

The 16bit format is the minimum format used by the ModBus protocol; the byte order used by the modbus protocol is big-endian, from the most significant byte to the least significant byte.

Formato DWORD 32bit

32bit data is stored in two consecutive registers starting from the even address.

	Address
HIGH REGISTER	0
LOW REGISTER	1

The byte order used by the ModBus protocol is big-endian, from most significant to least significant byte.

String format

String format fields are stored on consecutive registers, two bytes for each register; the maximum limit of registers read in block is 125 and the maximum limit of registers written in block is 123. The table below shows an example of how a 10 character ASCII string will be stored.

Es: String “0123456789”

	Address	Value
REG.	0	0x3031
……..	1	0x3233
……..	…	0x3435
		0x3637
REG.	4	0x3839

The byte order used by the Modbus protocol is big-endian, from most significant to least significant byte.

13. Log

The feature allows you to trace commands and data coming from an external supervisor

Enable

In the Ppg.ini file under the [APP] section add the keys

IOT_LOG=1

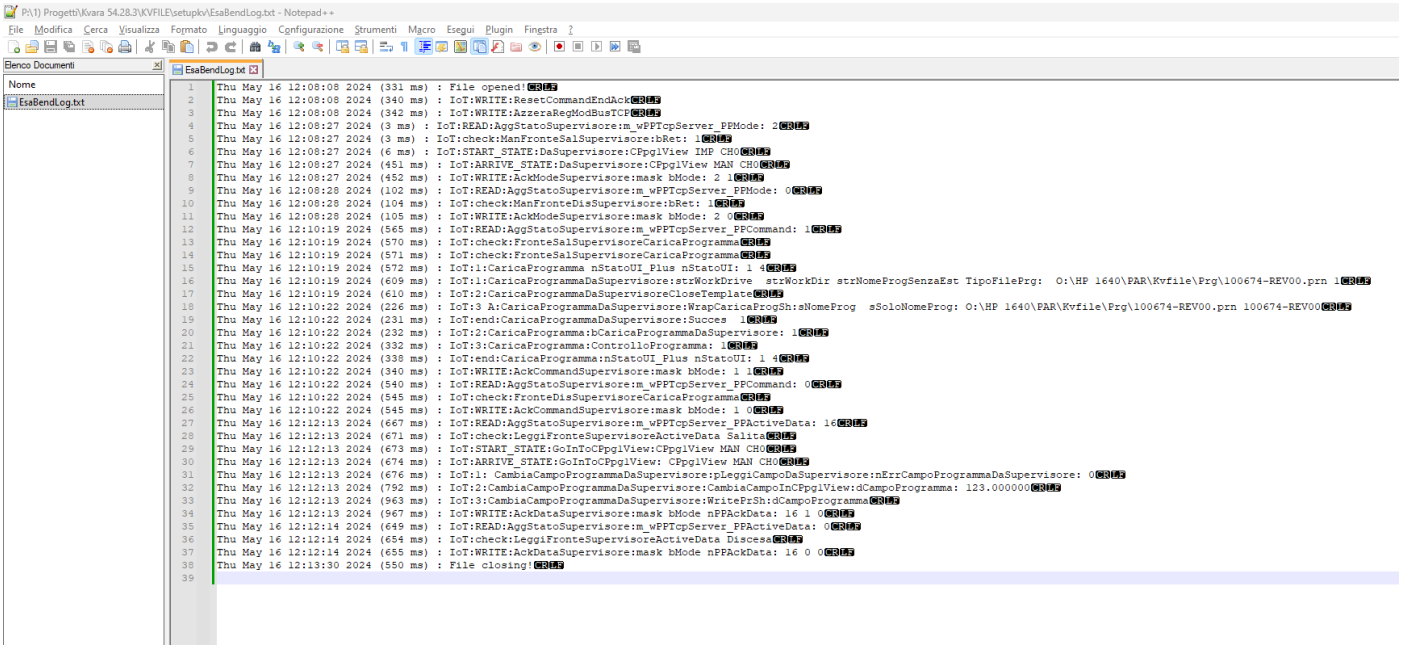
LOG_FILE=1

Log file

In

C:\KVFILE\setupkv\ EsaBendLog.txt

there is log file



The Log file is reset every time EsaBend is restarted; in case of malfunction, copy and send EsaBendLog.txt to Esautomotion BEFORE restarting EsaBend.

14. Test con Client EsaBendClient.exe

To perform connection, command, data writing and data reading tests from clients, the EsaBendClient.exe application can be used, see EsaBendClient- Rev.0 - ENG.pdf document.

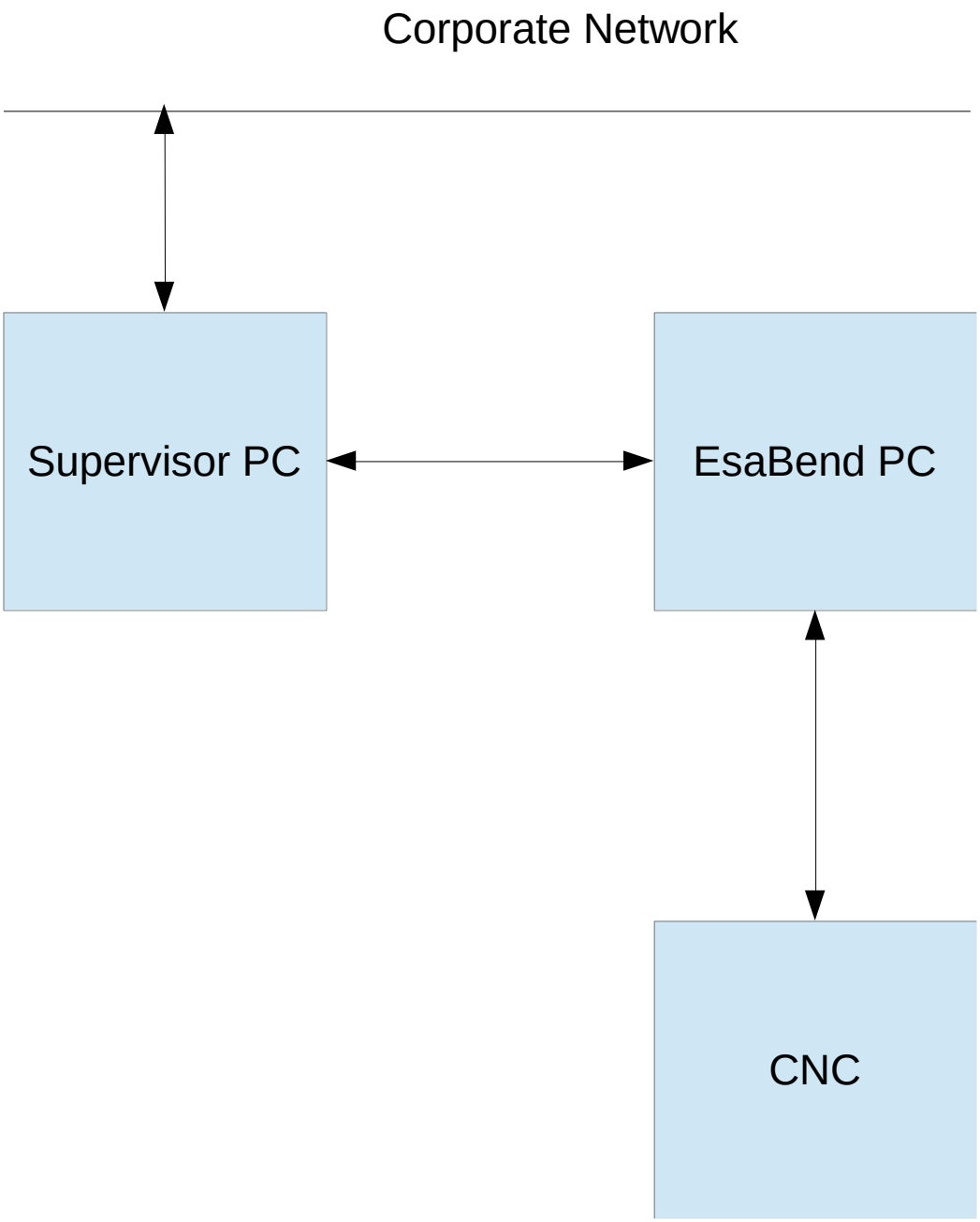
15. EsaBend ModBusTCP connection configuration

1. Description

The connection configuration between supervisor and Esabend ModBusTCP server is in the ServerModBus.XML file; the default written configuration is localost (127.0.0.1), i.e. the supervisor on the same PC as EsaBend. If you want to modify it, you need to change the ServerModBus.XML file and the window network configuration.

2. *Supervisor PC physical connection with EsaBend PC*

1. Direct connection



- For EsaBend PC configuration use the following IP addres, IP address: 172.16.0.51, Subnet mask: 255.255.0.0
- EsaBend PC in C:\Kvara\exe\ServerModBus.XML set the key <IPAddr>172.16.0.51</IPAddr>
- For Supervision PC configuration use the following IP addres, IP address: 172.16.0.50, Subnet mask: 255.255.0.0

To configure: Start->Network Connections (Fig.1)

Fig.1

Right click on Len3 (or Len 2) Fig.2

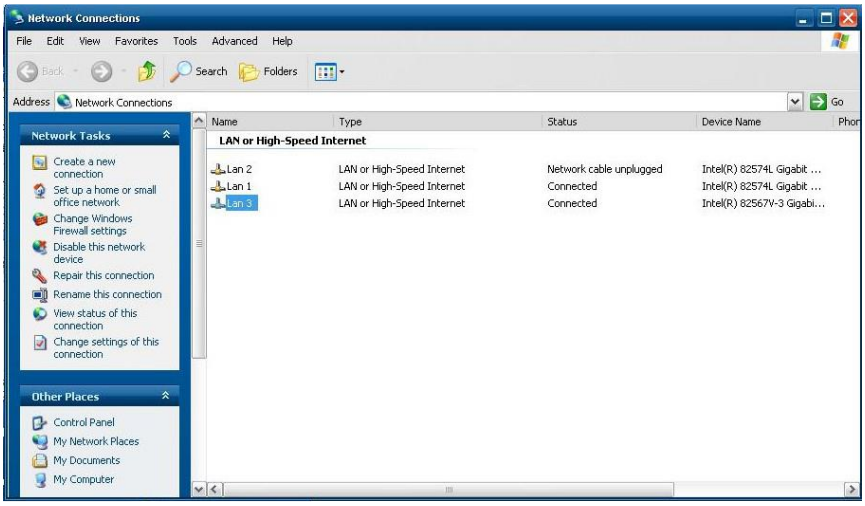


Fig.2

General, Internet Protocol (TCP\IP), Properties (Fig.3)

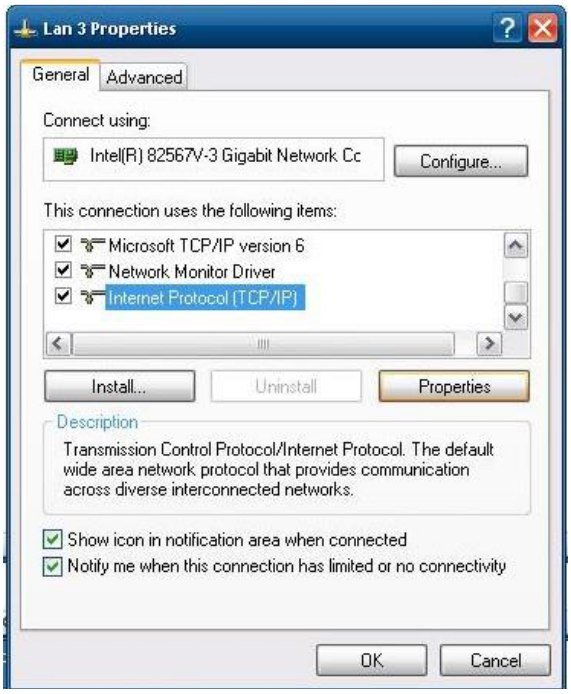


Fig.3

• For EsaBend PC configuration use the following IP addres, IP address: 172.16.0.51, Subnet mask: 255.255.0.0 (Fig.4)

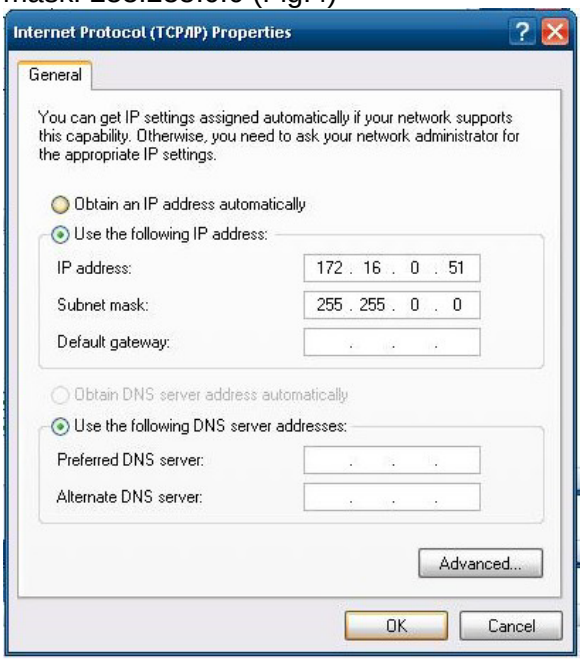


Fig.4

• For Supervision PC configuration use the following IP addres, IP address: 172.16.0.50, Subnet mask: 255.255.0.0 (Fig.5)

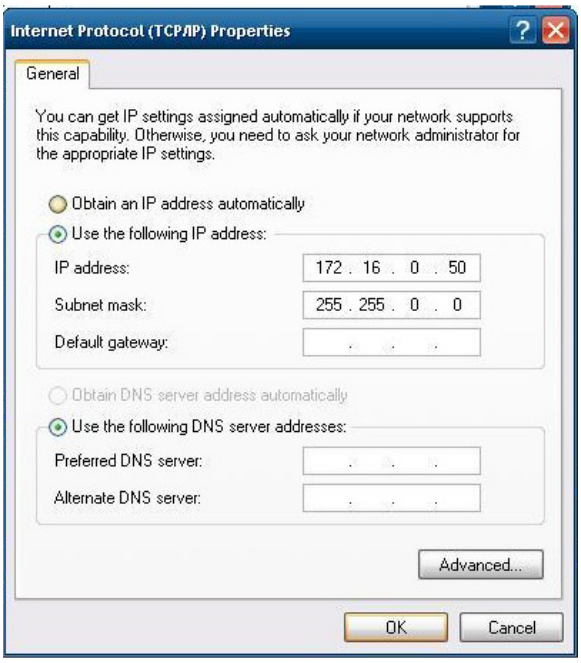
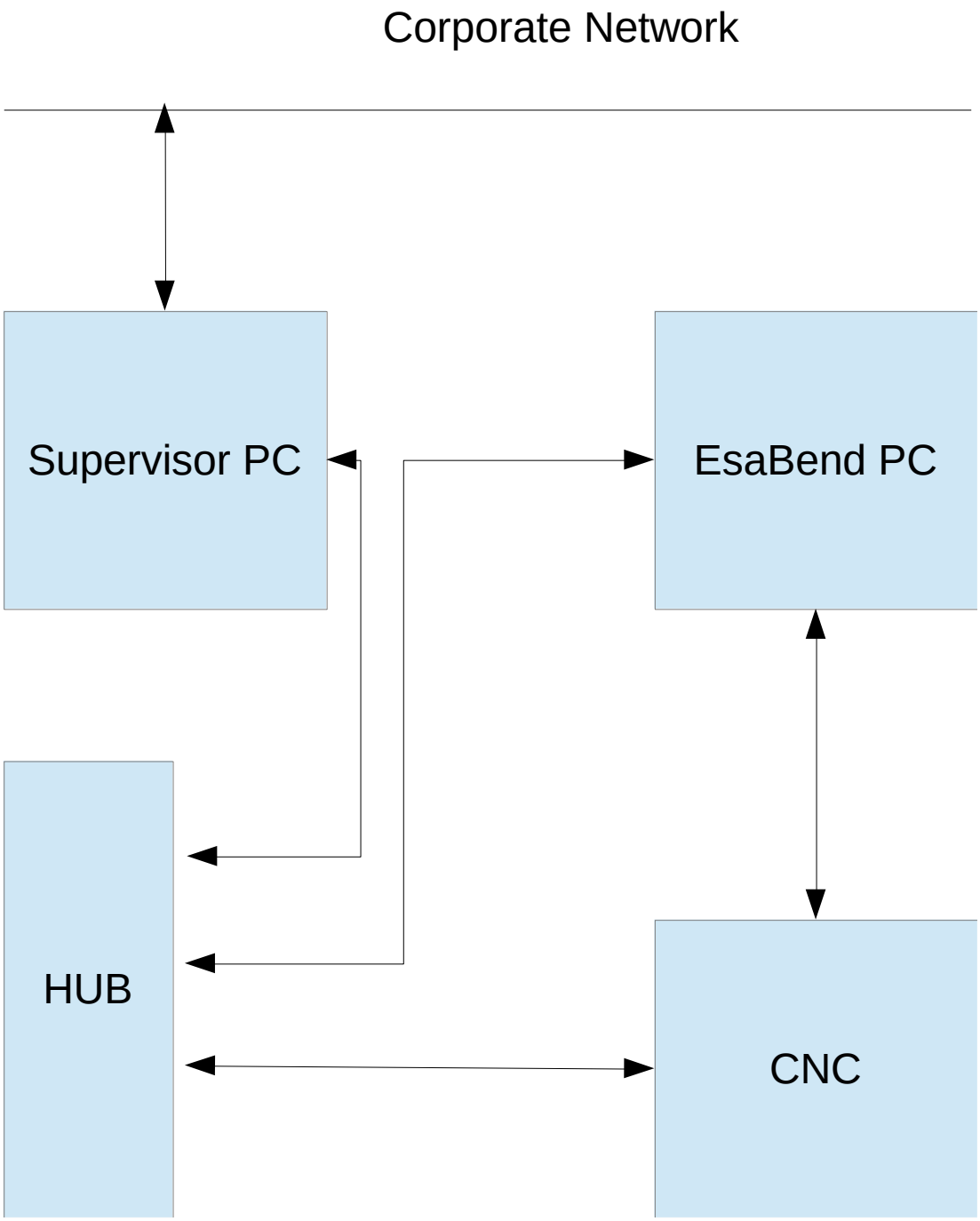


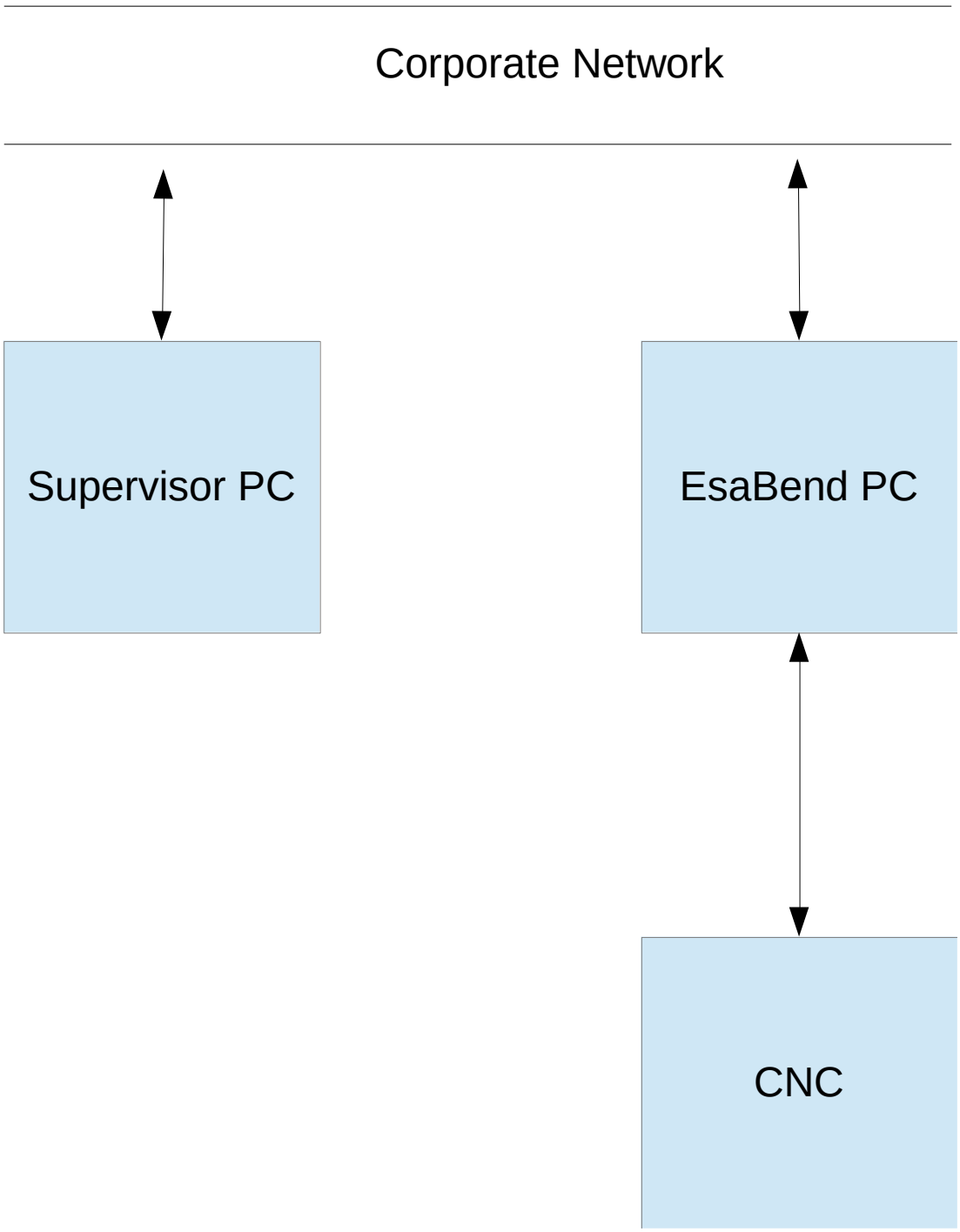
Fig.5

2. Direct connections with hub

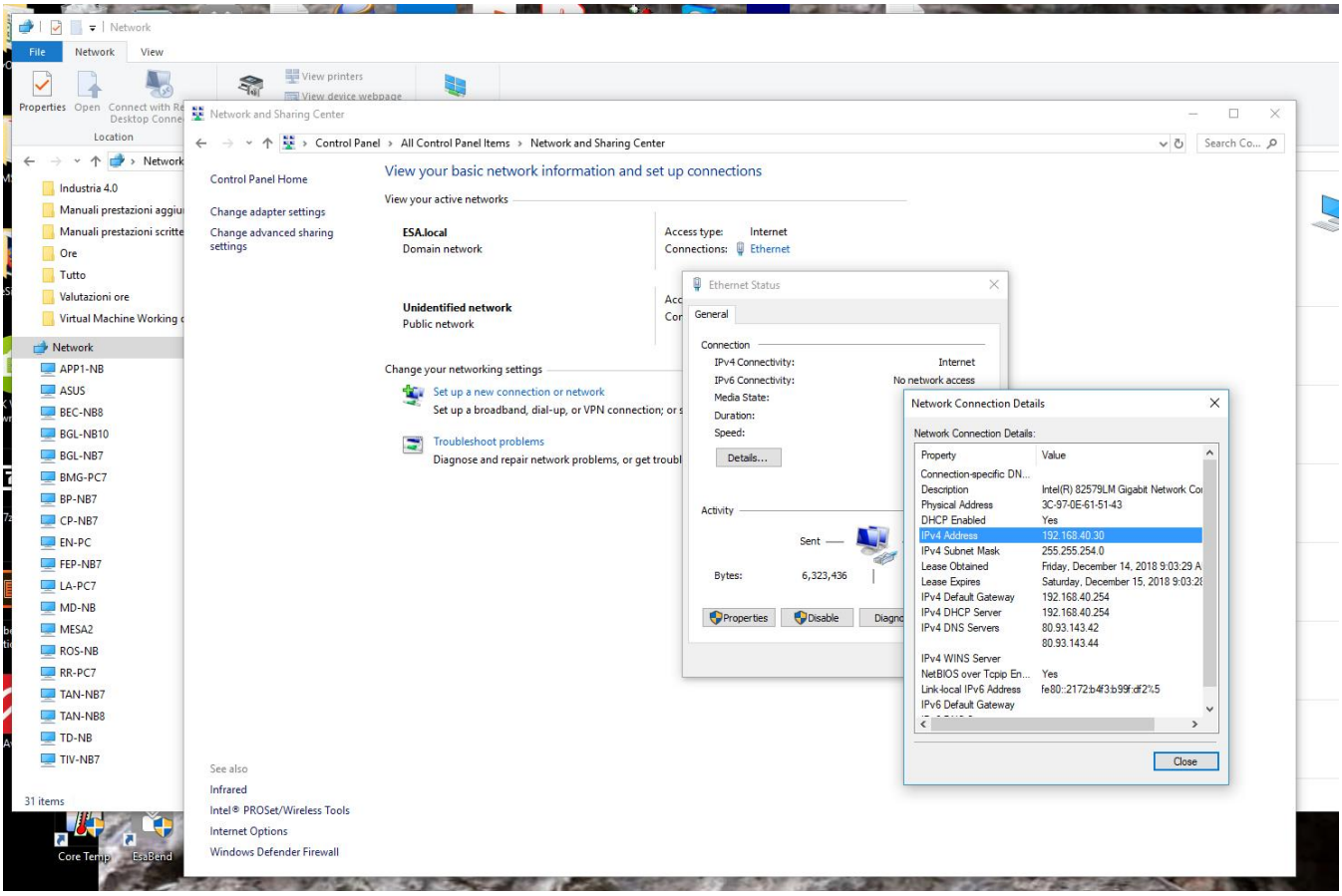


• For EsaBend PC configuration use the following IP addres, IP address: 10.10.51.99, Subnet mask: 255.255.0.0
• EsaBend PC in C:\Kvara\exe\ServerModBus.XML set the key <IPAddr>10.10.51.99<IPAddr>
• For Supervision PC configuration use the following IP addres, IP address: 10.10.51.51, Subnet mask: 255.255.0.0

3. Connection with corporate network



To set the correct IP in C:\Kvara\exe\ServerModBus.XML check:

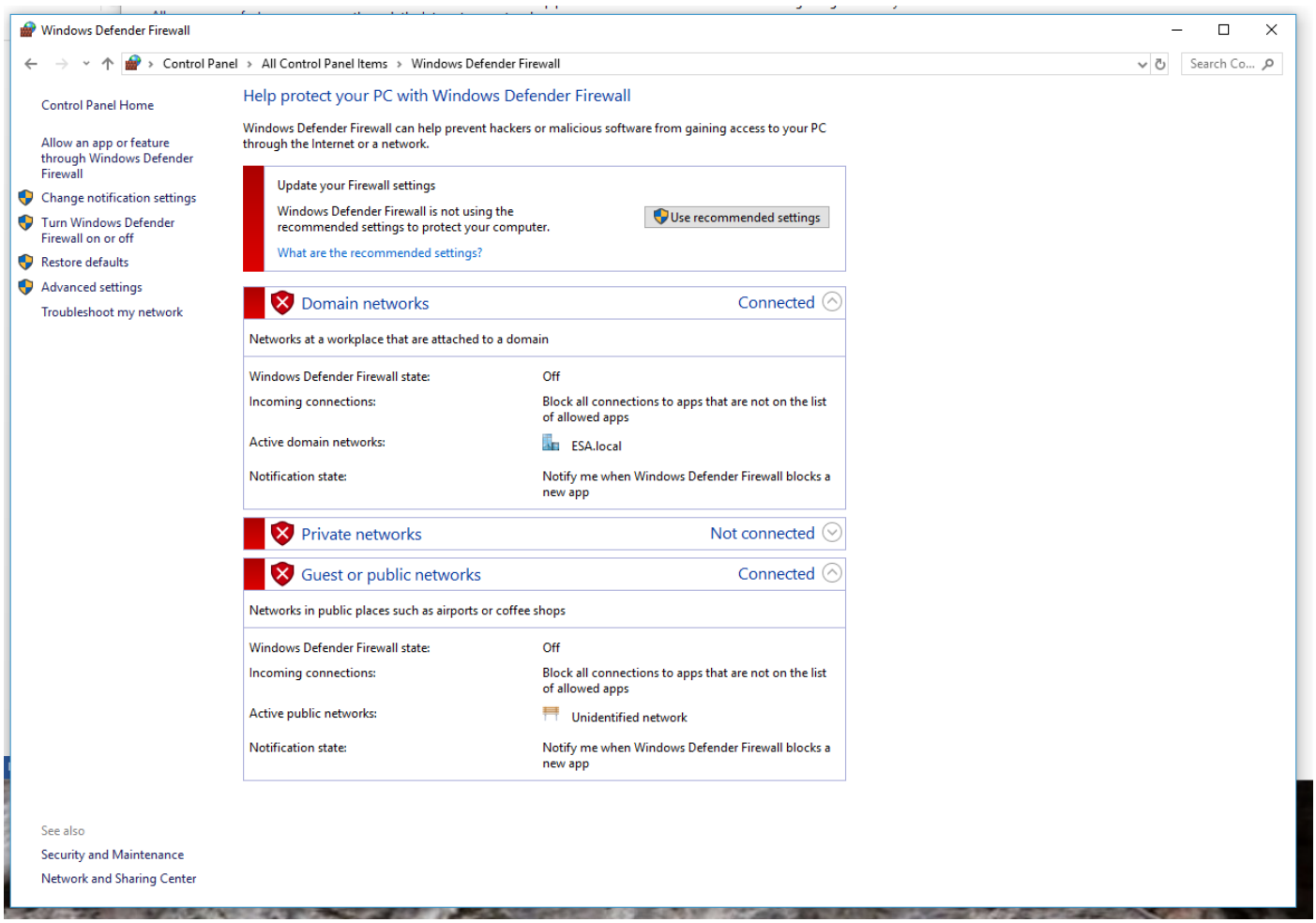


- For EsaBend PC configuration use the IP addres, where the first two numbers of the address are the same corporate network address. This static IP must be set also in the corporate server (ex. 192.168.40.30)
- In EsaBend PC in C:\Kvara\exe\ServerModBus.XML set the key <IPAddr>192.168.40.30<IPAddr>
- The Supervision PC configuration IP addres is dynamically assigned by the corporate server
- The Supervisor Client must connect (in this example) 192.168.40.30

16. Antivirus

Many antiviruses block the channel that uses ModBusTCP so it is necessary to either configure the antivirus so that it does not block communication or completely stop the antivirus.

Also disable the "Windows Defender Fire Wall" on the PC where EsaBend resides:



15. Directly automatically

When loading the program from the supervisor, EsaBend goes directly into automatic mode.

If you don't want to log in automatically, you need to set the key below in PPG.ini

[ModBusTCPClient]

NO_AUTO=0

Andrea Turchi

Rev.2 09/08/2024

Rev.1 08/05/2024

Rev.0 26/06/2024